

SaSSy CRM

From first login
to reliable automation.

Setup, daily operations, team access, and integrations
with a complete Make.com scenario-creation chapter.

01 GET STARTED

Accounts, workspaces, and the daily CRM workflow

02 OPERATE

Supabase, Cloudflare, teams, and platform care

03 AUTOMATE

Make, WhatsApp Business API, and Brevo email

A practical guide, with implementation boundaries clearly marked.

Existing CRM features and future delivery services are documented separately.

How to use this handbook

SaSSy CRM is a configurable customer relationship management application for managing enquiries, contacts, opportunities, follow-ups, and team activity. It supports multiple business workspaces in a shared SaaS installation.

This guide uses the product name **SaSSy CRM**. Some source files, application labels, package names, and the existing deployment still use **Workspace CRM** or a business's chosen name. This documentation does not rename the deployed application or change its URL.

Read this first: Core CRM functionality is implemented. General-purpose WhatsApp Business API delivery, Brevo CRM email delivery, scheduled campaigns, and subscription billing are not included as finished services. Their setup sections below are clearly marked as implementation guides. Adding API keys alone will not enable them.

The Make.com creation walkthrough is included in chapter 15 of this handbook, covering lead capture, WhatsApp/Brevo orchestration, scheduling, duplicate prevention, and recovery.

The capability descriptions were checked against the source in this workspace. Historical deployment notes are not a fresh verification of live cloud settings, email delivery, or third-party scenarios. No production configuration, account, customer record, or automation was changed to create this guide.

Find your route

Business users: chapters 3-6. Platform operators: chapters 7-11 and 17-19. Automation builders: chapters 12-16, including the complete Make creation walkthrough in chapter 15.

The contents page and PDF bookmarks are clickable. Web references open external documentation. Repository source references retain their filenames; they are not public download links. Examples and placeholders must be reviewed before running commands or enabling live delivery.

Contents

20 chapters, with the Make.com walkthrough integrated in chapter 15.

1. Choose your starting point	4
2. What is available today	5
3. Create an account and workspace	7
4. Use the CRM every day	8
5. Configure Settings	10
6. Manage team access	11
7. Run the project locally	12
8. Connect a new Supabase backend	13
9. Set up authentication emails with Brevo	15
10. Build and host on Cloudflare Pages	16
11. Connect the optional Cloudflare gateway	18
12. Understand the automation architecture	20
13. Implement WhatsApp Business API automation	22
14. Implement Brevo CRM email automation	24
15. Create Make automations	26
16. Example automation recipes	36
17. Operate and maintain the platform	38
18. Troubleshooting	40
19. Verification and launch checklist	42
20. Developer map and references	44

CHAPTER 01

Choose your starting point

Your role	What you need to do	Start here
Business owner using the hosted CRM	Create an account and workspace, customize it, invite colleagues	Sections 3-6
Invited team member	Verify the invited email, accept the invitation, work assigned enquiries	Sections 3, 4, and 6
Platform operator hosting SaSSy CRM	Configure the shared database, authentication, hosting, and operations	Sections 7-11 and 17-19
Developer adding messaging or automation	Implement trusted provider connections and durable workflows	Sections 12-16 and 20

Important distinctions

- **Account:** a person's sign-in identity and password.
- **Workspace:** a business's records, settings, and team membership.
- **Workspace Owner:** manages that business; this does not grant access to Cloudflare or the shared database administration console.
- **Platform operator:** maintains the infrastructure serving all workspaces.
- **Connection:** the database or integration endpoint used by the application. Saving a connection does not create an account.
- **Automation:** a backend process that reacts to an event or schedule. A visible switch, template, or settings page does not prove such a process exists.

In the hosted SaaS model, the operator connects infrastructure once. Each customer does **not** need their own Supabase, Cloudflare, or database account to use core CRM features.

CHAPTER 02

What is available today

Capability	Current status	Practical boundary
Leads and customer records	Implemented	Contact details, business information, priorities, ownership, notes, and activity history
Visual sales pipeline	Implemented	Stage columns, owner filtering, and stage changes through a selector; not an arbitrary drag-and-drop workflow designer
Reminders and calendar	Implemented	In-app follow-ups and meetings; external reminder delivery requires a backend
Analytics and daily activity	Implemented	Based on available CRM records; not a financial accounting or attribution platform
Multiple workspaces	Implemented in SaaS mode	Membership-scoped data and settings; one person can belong to several businesses
Team invitations	Implemented in SaaS mode	An Owner creates a private link and shares it manually; invitation email is not automatic
Business customization	Implemented	Industry presets, terminology, stage labels, currency, calling code, and custom fields
Appearance	Implemented	Light, dark, device mode, six palettes, and custom colors; personal browser preference
Mobile calling	Implemented	Opens the device's dialer; does not place, record, or automatically log a completed call
Spreadsheet import/export	Implemented with limitations	Imports create records; exports are not a full database backup; review the dependency warning below
Recycle bin	Implemented	Soft-deleted leads can be restored; permanent deletion cannot be undone in the CRM
Templates and integration screens	Implemented	Content and controls do not install provider services
Cloudflare gateway	Included, requires operator deployment	Connection, authentication, and workspace-access checks only
WhatsApp/Brevo CRM delivery and campaign execution	Additional implementation required	Provider adapters, authorization, consent, webhooks, jobs, and delivery storage are needed
Business-specific Meta lead intake	Separate extension documented	A historical Make -> Supabase -> Gmail workflow exists for one business; not automatic onboarding for every workspace
Billing, subscriptions, quotas, platform-admin console	Not implemented	Required separately if commercializing the product

Scope and limitations

- SaaS is an architecture here, not a claim that the full commercial operating platform is complete.
- The database integration is specifically Supabase/PostgreSQL. Connecting an arbitrary database requires development; Settings is not a universal database connector.
- Changing a stage's display label preserves its underlying stage and won/lost meaning. It does not add or reorder the canonical workflow.
- Opportunity values are estimates. Totals are grouped by currency; there is no automatic currency conversion.
- An installable app shell is not offline synchronization of live CRM records.
- The healthcare/wellness preset is for enquiries and scheduling, not regulated medical records.
- Existing dependency advisories, especially around spreadsheet parsing, must be reviewed and resolved before relying on untrusted imports. See the dated warning in the README; its historical counts are not a fresh audit.

CHAPTER 03

Create an account and workspace

3.1 Start as a business owner

1. Open the website supplied by your platform operator.
2. Choose **Create an account**. There is no default email or password.
3. Enter your own account details and choose a strong, unique password.
4. Open the verification email and follow its link.
5. Sign in if prompted.
6. Choose to create a workspace, enter your business details, and complete the setup.
7. Review **Settings -> Workspace**, then invite colleagues through **Settings -> Team & access**.

Creating an account alone does not grant access to someone else's business records. The first person creating a workspace becomes its Owner.

If verification emails do not arrive, contact the platform operator. Do not repeatedly create accounts, change databases, or disable verification. Production authentication email delivery must be configured by the operator; see section 9.

3.2 Join an existing business

1. Ask the workspace Owner for an invitation addressed to your exact email.
2. Open the private invitation link.
3. Create an account or sign in using that email, and verify it.
4. Accept the invitation and open the workspace.

An account under a different email cannot accept the invitation. Expired or revoked links must be replaced by the Owner. Do not create a second workspace just because an invitation has not been accepted yet.

3.3 Switch businesses

Use the workspace switcher, or **Settings -> Switch workspace** where available. Switching reloads the application so old records and editors are not retained in the new business context. Save your work first.

Your role may differ between workspaces. Losing access to one workspace does not delete your account or memberships in other workspaces.

CHAPTER 04

Use the CRM every day

4.1 A practical daily routine

1. Confirm that you are in the correct workspace.
2. Review **Reminders** for due or overdue follow-ups and **Calendar** for meetings.
3. Review new and unassigned leads. Assign a responsible teammate where permitted.
4. Open a lead, review its history, and contact the person through an appropriate channel.
5. Record what happened, update the qualification or stage, and set the next follow-up.
6. Review **Pipeline**, **Analytics**, and **Daily** to identify stalled opportunities and missing activity.

Do not mark a message delivered or a call completed merely because you clicked a button.

4.2 Add and maintain a lead

Open the leads screen; its label may be **Enquiries**, **Prospects**, **Customers**, or another workspace term. Choose the add-record action.

Capture the details relevant to your business:

- Name and available phone/email details.
- Company, role, location, website, and product or service interest.
- Source/category, qualification, priority, and sales stage.
- Responsible teammate, next follow-up date/time, and notes.
- Opportunity value, currency, expected close date, and configured custom fields.

Use a full international phone number when it is known. Do not guess a customer's country or fabricate contact details. An email address or phone number is not, by itself, permission for marketing.

After a conversation, add an activity note with an accurate outcome and next action. Keep sensitive or unnecessary personal information out of free-text notes.

4.3 Search, filter, and prioritize

Use search and filters to narrow the list by relevant fields such as stage, qualification, owner, priority, category, and follow-up date. Review filters before deciding records are missing. A different workspace, filter, or sort order can change what you see without deleting anything.

Assignment helps organize work; it is **not** a private-record security boundary. Authorized staff may access ordinary CRM records in their workspace even when those records are assigned to another teammate.

4.4 Use the pipeline

Open **Pipeline** to see records grouped by stage. Filter by owner or unassigned records. Open a record from its card, or use its stage selector to update its position if your role allows editing.

The overview separates open potential value and won value. Mixed currencies remain separate. Renaming "Closed Won" to "Enrolled" or "Deal closed" changes business language, not the underlying successful-outcome meaning.

4.5 Meetings and follow-ups

Use the lead profile and calendar to manage appointments and follow-ups. Review meeting status after the event and reschedule when needed.

In-app reminders can be used now. Automatic WhatsApp/email reminders, and reliable notifications while the app is closed, need their corresponding backend delivery services. A follow-up date alone does not schedule an external message.

4.6 Call from a smartphone

On supported mobile layouts, a **Call** action appears on lead cards and profiles when the saved phone number is usable.

1. Tap **Call**.
2. Review the number in the device's dialer.
3. Choose whether to place the call.
4. Return to SaSSy CRM and log the outcome yourself.

This is a normal telephone link, not a telephony service. Call charges are handled by your phone provider. Missing or unsafe numbers do not produce a call action, and an unsaved phone edit is not used as the saved contact number.

4.7 Import a spreadsheet

Use trusted files only until the documented spreadsheet dependency issues are resolved.

1. Confirm the destination workspace and create any required custom fields first.
2. Choose **Import** from the lead dashboard.
3. Select a .csv, .xlsx, or .xls file.
4. Review column mappings. A column must map to **Name**; map each CRM field only once.
5. Skip unrelated columns and inspect the preview.
6. Use valid dates, typically YYYY-MM-DD, and valid numeric values. Confirm stage/category mappings.
7. Import a small test batch before a large real dataset.
8. Review the saved records and then proceed with the remaining data.

Rows are validated before writes begin, but network failures can still interrupt saving. If the dialog reports that some rows were saved, use its continuation action to retry the remaining rows while the same dialog is open. Closing/reopening it or importing the whole file again can duplicate already saved records.

Imports create new records; they are not an upsert, deduplication service, full restore, or transfer of consent evidence. Importing a contact does not grant WhatsApp or email marketing consent.

4.8 Export and restore records

An authorized user can export the currently filtered list in its current order to an Excel workbook. Check filters first if you want a wider export. Exported information can include activity details depending on the available data.

Treat exported files as confidential customer data. An export does not include everything needed to restore authentication, memberships, settings, provider configuration, or a whole database.

Deleting a lead normally moves it to **Bin**. Authorized users can restore it. **Delete forever** and **Empty Bin** are destructive Owner actions; do not use them as ordinary cleanup or assume they can be reversed through the application.

CHAPTER 05

Configure Settings

Sections are shown according to your role. A hidden section can be a permission restriction rather than an application error.

Section	Purpose	Notes
Overview	Review the setup checklist and current workspace	A completed UI step is not an integration delivery test
Workspace	Business identity, industry, terminology, currency, stages, custom fields	Owner/Manager configuration; saved for the workspace
Connections	Inspect/test database and optional gateway	Shared infrastructure is normally locked in hosted SaaS
Team & access	Invitations and member access	Only Owners issue SaaS invitations
Intake	Categories and form-to-category mappings	Saving a mapping does not publish a form or start webhook intake
Appearance	Display mode and accent colors	Applies to this browser; does not change teammates' preferences
Account	Account details and password-related controls	Personal account is separate from workspace membership

Workspace customization

Available presets include general business, professional services, real estate, education/training, retail/commerce, and healthcare/wellness enquiries.

Choose a suitable preset, then review its labels and fields. Custom fields support text, number, date, and dropdown values. Keep names clear and avoid collecting data you do not need. The current form supports up to 50 custom fields; dropdown choices are also bounded.

Custom field keys are important to imports and integrations. Coordinate changes with whoever maintains those mappings. Removing a field definition is not a verified database erasure procedure for historical values.

Appearance

Choose **Light**, **Dark**, or **Follow device**. Accent options are Blue, Teal, Violet, Rose, Amber, Graphite, and a custom color. Use the picker or a valid hexadecimal color and apply it.

These preferences persist in browser storage when available. They are not account-wide theme synchronization. Clearing browser data may remove them, and another browser/device can use a different theme.

Connections are not customer credentials

A hosted deployment normally locks shared database settings. Workspace Owners should not replace the database used by every customer.

In an unlocked operator/development installation, changing a connection requires testing and confirmation. Saving signs out the old connection and reloads this browser; it does not copy, migrate, or delete records. A browser-local setting does not configure other visitors.

Never enter a Supabase service-role/secret key, database password, Cloudflare API token, Brevo API/SMTP key, or WhatsApp token into the frontend connection form.

CHAPTER 06

Manage team access

Roles

The table summarizes the current UI permissions, not a promise of per-record privacy or a substitute for backend authorization.

Role	Typical responsibilities	Important restriction
Owner	Workspace configuration, invitations, member management, all CRM operations	Workspace ownership is not platform infrastructure administration
Manager	Daily operations, configuration, export/deletion, management of lower staff roles	Cannot invite in SaaS, manage Owners/other Managers, or change their own access
Sales	Lead work, reminders, calendar, analytics, message controls where connected	No workspace/team administration or export/delete controls
Support	Lead capture, follow-ups, calendar, message controls where connected	No analytics or workspace/team administration controls
Marketing	Lead work, analytics, intake/category work, templates	No team administration or direct send permission in the current UI

Provider-related permissions do not create a provider connection. The backend must independently enforce each allowed action.

Invite a teammate

1. As an Owner, open **Settings -> Team & access**.
2. Enter the teammate's email and choose Manager, Sales, Support, or Marketing.
3. Choose **Create invitation**.
4. Copy the private link immediately and share it securely with that person.
5. Ask them to sign in with the matching verified email and accept it.

No invitation email is sent by this action. The raw link cannot be retrieved after dismissal or reload. Invitations expire after seven days, can be revoked, and can be accepted once. The invitation form does not create another Owner.

Change or remove access

Use **Edit access** on a member you are permitted to manage. Review the role and active-access setting before saving.

Disabling membership removes access to this workspace while retaining the person's account and activity history. It does not disable their other business memberships. Users cannot change their own role through this flow. Access updates check for conflicting versions; refresh and review again if another administrator edited the same member.

When someone leaves, disable their workspace access, revoke pending invitations, and separately review any provider, Make/n8n, hosting, or database access they held outside the CRM.

CHAPTER 07

Run the project locally

Audience: operator or developer. These steps are not required for ordinary hosted customers.

Prerequisites

- A copy of the project and a terminal in its `crm` directory.
- Node.js and npm compatible with the locked dependencies, including the separately installed Cloudflare tooling if used. Prefer a currently supported Node LTS and check package engine warnings.
- A browser. Live operation additionally needs a separately configured Supabase project.

The examples below use PowerShell and `npm.cmd` to avoid Windows script-execution-policy issues. On macOS/Linux, use `npm` and adapt filesystem paths.

Explore in demo mode

Review `.env` in an editor. A deliberate demo configuration is:

DOTENV

```
VITE_CRM_MODE=demo
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
VITE_WORKER_URL=
VITE_ALLOW_CONNECTION_SETUP=false
```

From the CRM directory:

POWERSHELL

```
npm.cmd ci
npm.cmd run dev -- --host 127.0.0.1
```

Open the local address printed by Vite. The configured starting port is 5174; the terminal is authoritative if that port is occupied.

Use a fresh browser profile or inspect any previously saved local connection: in an unlocked/demo installation, an existing browser connection can take precedence over the file. Confirm the application says **Demo mode** before experimenting.

Demo records and messages are simulated. Records reset on reload; some browser preferences persist. Do not use demo mode to store business data or validate real email delivery. Keep the development server local; it is not a production host.

The root `.env.production` supplies production build values and may point at the existing hosted installation. Review it before building for a different business or staging environment. Keep all private credentials out of source control; do not assume a local file is ignored simply because its name begins with a dot.

CHAPTER 08

Connect a new Supabase backend

Audience: platform operator. Use this installation procedure only for a new, empty database.

8.1 Select the operating mode

Mode	Intended use	Database installation
demo	Local evaluation with simulated data	None
saas	Multiple isolated business workspaces in one deployment	supabase/saas.sql
supabase	A dedicated, single-business installation	supabase/standalone.sql

Use **SaaS** for the multi-business product described in this guide. The two SQL files are alternative installations, not sequential migrations.

Do not run both schemas, rerun the full schema to fix sign-in, or apply archived SQL to fill missing features. An existing project with no leads can still contain accounts, tables, and configuration. Inspect it first with `setup-check.sql` and plan a migration when needed.

8.2 Create and initialize the project

1. Create a new Supabase project in the intended operator organization.
2. Review the target project identity and the SaaS schema.
3. Run that schema once in the new project's SQL Editor.
4. Confirm successful completion and review table access policies.
5. Run this read-only compatibility check:

SQL

```
select public.crm_schema_info();
```

The expected marker for this source version is SaaS mode, version 1. A successful authentication endpoint alone does not prove the CRM schema is installed.

Core data includes accounts/profiles, workspaces, memberships, workspace settings, invitations, leads, activities, appointments, templates, automation definitions, messages, notifications, categories, and form mappings. The presence of automation/message tables does not mean a scheduler or sender is installed.

8.3 Configure authentication

1. Enable email/password signup and email confirmation.
2. Set the Supabase **Site URL** to the actual HTTPS CRM address.
3. Allow the exact verification/recovery redirect URL used by that deployment.
4. Configure production SMTP delivery; see section 9.
5. Review signup and email rate limits, abuse protection, and recovery behavior.

For the existing project URL, the intended website address is `https://vertex-workspace-crm.pages.dev/`. A separate installation or staging site needs its own correct URL. Do not leave production verification links pointing to localhost or use a broad wildcard merely to avoid configuring the right address.

8.4 Supply public frontend configuration

For a hosted SaaS build, configure these values in the production build environment or the production environment file used by your local build:

DOTENV

```
VITE_CRM_MODE=saas
VITE_SUPABASE_URL=https://YOUR-PROJECT.supabase.co
VITE_SUPABASE_ANON_KEY=YOUR-PUBLISHABLE-KEY
VITE_WORKER_URL=
VITE_ALLOW_CONNECTION_SETUP=false
```

Replace the placeholders. The variable retains the name `VITE_SUPABASE_ANON_KEY`, but accepts the project's publishable key or supported legacy anon key. Keep the Worker URL blank until the gateway or a compatible integration service is ready.

Every `VITE_` value is browser-visible. Never place private database or provider credentials there. Rebuilding is required after changing build-time configuration; changing dashboard variables does not modify an already uploaded static bundle.

For an unlocked local setup, use **Settings -> Connections**, select SaaS, enter the public project URL/key, test, confirm the sign-out warning, and save. The first new SaaS connection leads into account creation. You must still choose your own email/password and verify it.

8.5 Verify isolation before onboarding customers

Create two synthetic test businesses with different accounts in staging. Confirm each can manage its own records and cannot read or modify the other's records, memberships, settings, or related activities. Test revoked membership as well as an ordinary successful sign-in.

The application sends a selected `x-workspace-id` and scopes queries; PostgreSQL row-level security checks active membership. A workspace header is not authentication. Server-side integrations using elevated privileges must explicitly reproduce appropriate isolation checks because elevated keys bypass ordinary RLS.

Standalone account provisioning differs and has no equivalent self-service workspace signup. See the standalone instructions if intentionally choosing that mode.

CHAPTER 09

Set up authentication emails with Brevo

Available as an operator configuration task; not performed by this guide. This can enable Supabase verification and password-reset emails without building the CRM campaign sender.

There are two separate Brevo connections:

Purpose	Sending path	Credential location
Account verification and password recovery	Supabase Auth -> Brevo SMTP -> account holder	Supabase Auth's private SMTP configuration
CRM reminders, transactional messages, or permitted campaigns	Trusted CRM integration backend -> Brevo API -> contact	Backend secret storage; requires additional implementation

Completing the first row does not complete the second. It also does not make workspace invitation emails automatic.

Operator setup

1. Create or use the intended Brevo account and verify a sender/domain you control.
2. Add the domain-authentication DNS records Brevo currently requests. Review any existing DNS records before changing them; do not overwrite unrelated mail configuration.
3. In Brevo, obtain the SMTP settings and create an **SMTP key**, not an API key.
4. In the intended Supabase project, open Authentication's custom SMTP settings.
5. Enter the Brevo SMTP host, port, login, SMTP key, sender email, and sender name. Use a TLS configuration supported by both services and their current documentation.
6. Save the SMTP configuration and recheck the CRM Site URL and redirect allowlist.
7. Test a signup and password recovery using addresses you control, including an address outside the Supabase organization. Verify actual receipt and the final browser destination.
8. Review delivery logs and rate limits before inviting customers.

Supabase's default email service is for restricted/testing use, including restrictions on recipients outside the project's organization. Earlier deployment notes record that custom SMTP was not configured; that historical note must be checked against the current dashboard before drawing conclusions about today's delivery.

Never publish SMTP keys, share passwords with teammates, or disable account verification as an email-delivery workaround. See [Supabase custom SMTP](#) and [Brevo SMTP integration](#).

CHAPTER 10

Build and host on Cloudflare Pages

Audience: platform operator. Publishing changes the public application. The commands below are instructions, not actions already performed.

Deployment procedure

1. Review the intended Supabase configuration, site identity, and current privacy content.
2. Run the verification appropriate to your changes; see section 19.
3. Build from the CRM root:

POWERSHELL

```
npm.cmd run build
```

4. If the separate deployment tooling is not installed, install its locked dependencies:

POWERSHELL

```
Push-Location .\cloudflare  
npm.cmd ci  
Pop-Location
```

5. Check the signed-in Cloudflare account and exact Pages project before publishing:

POWERSHELL

```
.\cloudflare\node_modules\.bin\wrangler.cmd whoami  
.\cloudflare\node_modules\.bin\wrangler.cmd pages project list
```

6. After deliberately selecting the correct account/project, publish the complete build. Replace YOUR_PAGES_PROJECT with the verified target:

POWERSHELL

```
.\cloudflare\node_modules\.bin\wrangler.cmd pages deploy .\dist --project-name YOUR_PAGES_PROJECT --branch main
```

The existing project is named `vertex-workspace-crm`; use that name only when intentionally updating that installation in its authorized account. A new installation should have its own deliberately selected project. If authentication is missing, use Cloudflare's normal sign-in flow; do not paste an account-wide token into CRM Settings.

Upload **only the complete dist output**, not the repository, SQL, environment files, customer exports, or just one edited page. Preserve required public assets, routing, and the correct privacy policy. Historical release notes contain a policy-only release; verify the current source/output rather than assuming every old build contains the newest policy.

Verify the release

- Open the deployment URL and intended production URL in a fresh browser context.
- Confirm correct branding/workspace context and SaaS sign-in, without customer infrastructure setup.
- Test verification/recovery redirects with a controlled account.
- Check lead operations and workspace isolation using synthetic staging records before broad rollout.
- Verify public privacy content applies to this product and deployment. A policy written for one customer is not automatically a platform-wide SaaS privacy policy.
- Confirm no private credentials or customer data entered public assets.
- Keep a known-good release and a rollback procedure. Frontend rollback does not roll back database migrations.

The existing Pages project is documented as **Direct Upload**, not automatic Git deployment. Do not assume pushing a repository publishes it. Adding Git-based delivery requires its own deployment design; consult the [current Cloudflare Direct Upload guide](#).

CHAPTER 11

Connect the optional Cloudflare gateway

Current capability: health and access checks only. Core CRM data and account operations connect directly to Supabase and do not require this Worker.

Configure and deploy

Review `cloudflare/wrangler.jsonc`. Its safe default configuration is intentionally unconfigured. Supply:

Variable	Value
SUPABASE_URL	The intended Supabase project's base HTTPS URL
SUPABASE_PUBLIC_KEY	Its publishable or legacy anon key, not an elevated key
ALLOWED_ORIGINS	Exact permitted frontend origins, separated by commas; no wildcard or path
ALLOW_LOCALHOST	Keep false in production

Select a deliberate Worker name and account. From `cloudflare/`, validate first:

POWERSHELL

```
npm.cmd ci
npm.cmd test
npm.cmd run check
npm.cmd run dry-run
```

After successful validation and a deliberate decision to publish to the verified account:

POWERSHELL

```
.\node_modules\.bin\wrangler.cmd deploy
```

Add the resulting base HTTPS Worker URL to `VITE_WORKER_URL` and rebuild the hosted frontend, or test/save it in an unlocked local Connections screen.

What a successful check proves

Route	Access	Meaning
GET /public/health	Anonymous	Expected gateway is reachable; no user or provider details
GET /health	Supabase bearer token and selected workspace	The user is authenticated and has active access to the selected workspace

The included gateway reports unavailable providers:

JSON

```
{
  "ok": true,
  "integrationsConnected": true,
  "database": true,
  "features": { "email": false, "whatsapp": false, "ai": "none", "push": false }
}
```

integrationsConnected: true is not evidence that WhatsApp, Brevo, AI, push, or a scheduler works. Other routes are not installed by deploying this gateway. See the gateway guide for response codes and validation details.

CHAPTER 12

Understand the automation architecture

Sections 12-16 describe additional integration work, except where explicitly identified as an existing business-specific extension. They are not a claim that these workflows are enabled.

Current architecture

TEXT

```
Browser: React/Vite CRM hosted on Cloudflare Pages
|
+--> Supabase Auth: accounts, sessions, verification, recovery
|
+--> Supabase PostgreSQL: CRM data and workspace access policies
|
+--> Optional Cloudflare gateway: connectivity and membership checks
```

Architecture to implement for messaging

TEXT

```
CRM action / verified intake event / scheduled trigger
-> Trusted backend: authenticate, authorize, resolve workspace
-> Durable job/outbox: validate consent, timing, template, and duplicates
-> Delivery worker: use the correct workspace's provider connection
    +-> Meta WhatsApp Business API
    +-> Brevo email API
-> Authenticated provider status webhook
-> Persist delivery history and show truthful CRM status
```

The frontend should request work; it should not own provider credentials or a reliable background scheduler. A browser tab closing must not lose scheduled jobs.

Required backend work

1. Implement the selected frontend API contracts, not just a generic URL.
2. Verify the Supabase session and current workspace membership/role on each user action.
3. Resolve every lead, template, provider connection, and job within the authorized workspace.
4. Add explicit migrations for required storage: provider connection references, consent evidence, durable jobs/outbox, webhook events, suppression, and run history.
5. Store secrets in a server-only secret store. Encrypt or reference per-workspace credentials; never return them in health responses.
6. Add retry-safe processing, locking/leases, unique idempotency keys, and safe handling of uncertain sends.
7. Receive and authenticate provider callbacks, deduplicate them, and record actual delivery outcomes.
8. Expose accurate provider health only after configuration and controlled tests succeed.
9. Add pause controls, monitoring, rate limits, auditing, and a documented recovery process.

Suggested new storage concepts above are a design checklist, not existing table names or a ready-to-run migration. The base schema does not install an outbox, scheduler, private attachment service, or general provider-secret management.

Frontend extension points

Purpose	Existing client contract	Implementation status
Provider status	GET /health	Gateway exists; provider readiness logic must be extended
Send a message	POST /send	Backend sender must be implemented; successful response includes a persisted message
Allow/pause a rule	POST /automation/toggle	Backend required; receives automation_id and enabled
Request a rule run	POST /automation/run	Backend and job execution required; receives automation_id and options
Sync approved WhatsApp templates	POST /whatsapp/templates/sync	Provider adapter required
WhatsApp consent/preferences	/leads/whatsapp-preference, /leads/whatsapp-consent, /leads/whatsapp-opt-out	Recipient-specific backend enforcement required
Quick replies	/whatsapp/quick-reply-rules, /whatsapp/quick-reply-runs	Rule execution/history backend required

Use the complete integration contracts and `src/lib/db.js` before coding. Live writes use a Supabase bearer token and, in SaaS, `x-workspace-id`. CORS must allow the exact frontend origin and the methods/headers actually implemented; the health-only gateway's narrow GET preflight is not sufficient for new POST routes.

The Automations UI is not a full arbitrary-rule designer. New rule creation/provisioning, schedules, provider mappings, and possibly UI controls must be designed alongside the backend. Advanced funnel API extension points are separate from the working visual sales pipeline.

Per-business provider ownership

For one controlled pilot, the operator can maintain one explicitly mapped provider connection. For a product used by unrelated businesses, decide how each business authorizes its own sender/account, how secrets are isolated, who pays usage charges, and how access is revoked.

Do not send one business's messages from another business's WhatsApp number or email domain. Customer self-service provider onboarding, OAuth/Embedded Signup, and billing attribution are separate features, not installed settings controls.

CHAPTER 13

Implement WhatsApp Business API automation

Status: development and provider onboarding required. The CRM's mobile **Call** button is unrelated to this integration.

13.1 Prepare the Meta account

1. Use the intended business's Meta account/assets and obtain permission to manage them.
2. Create/configure the appropriate Meta app and WhatsApp Business Platform setup using Meta's current onboarding instructions.
3. Configure the business phone number and the relevant WhatsApp/messaging account identifiers.
4. Use test assets first, then provision production access with the permissions and business verification Meta requires for the chosen use case.
5. Plan proper credential expiry/rotation. A temporary developer test token is not a production credential strategy.
6. For onboarding other businesses, review Meta's current partner/app-review/Embedded Signup requirements. Copying one operator token to every workspace is not a SaaS onboarding solution.

Keep identifiers separate from secrets: phone/account IDs identify assets; access tokens and app secrets authorize access and remain server-side.

13.2 Build the sender

Implement `/send` and any template/consent routes needed by the UI. The backend must:

- Check the user's action permission and selected workspace.
- Load the recipient and provider connection from that workspace, not from arbitrary browser-supplied destinations or tokens.
- Validate the recipient's international phone number and recipient-specific consent/suppression state.
- Validate message type, permitted template, language, parameters, and any attachments.
- Recheck eligibility immediately before dispatch, not only when the job was scheduled.
- Preserve the client's request ID for deduplication and persist a durable message/job record.
- Call Meta's documented message endpoint using a supported API version and the authorized business phone/account context.
- Persist the provider message identifier and return the saved CRM message expected by the frontend.

Keep full national calling information; do not silently guess a country or assume every phone number supports WhatsApp.

13.3 Templates and the service window

Meta distinguishes service messages within an open customer-service window from approved template messages used outside it. Its current documentation describes a 24-hour service window opened/reset by qualifying user contact. Validate the current rules and supported event types against the provider documentation and your implementation.

For a practical appointment reminder:

1. Create a template in the correct Meta business account.
2. Select the appropriate category/language and provide the required sample parameters.

3. Wait for approval and check its status before use.
4. Implement template synchronization into the selected workspace's CRM templates.
5. Map parameters such as first name, date, time, and business name; validate missing values.
6. Send a controlled test to a permitted recipient before enrolling customers.

Creating a template in SaSSy CRM does not submit it to Meta or make it approved. An "approved" status from another business's account is not authorization to use it in this workspace.

13.4 Receive inbound messages and delivery events

Implement a public HTTPS webhook endpoint specifically for Meta callbacks. Complete Meta's verification handshake and validate actual event authenticity using Meta's current signature rules. A successful handshake is not validation of every subsequent event.

Route events using trusted, registered business assets and persisted provider message IDs. Do not trust a workspace ID included by an unknown sender. Deduplicate repeated events and preserve inbound messages or delivery events with the correct workspace/lead relationship.

An API acceptance response is not proof of delivery. Track provider-reported sent/delivered/read/failure states where available; do not invent read status when it is unavailable. Process opt-outs promptly and exclude suppressed recipients from all pending work.

13.5 Release checklist

- Valid and invalid signatures; duplicate and out-of-order events.
- Wrong-workspace token, lead, template, and provider mapping rejected.
- Expired token, unavailable template, invalid recipient, and closed service window handled safely.
- Unsubscribe/opt-out before dispatch blocks a queued message.
- Retry does not create another send for the same logical action.
- Ambiguous provider timeout enters reconciliation, not an automatic blind resend.
- Pause stops pending dispatch even if a run has already been scheduled.

See [Meta getting started](#) and [message/service-window guidance](#). Provider policy, account requirements, versions, and pricing can change; do not infer unlimited or free messaging from this CRM.

CHAPTER 14

Implement Brevo CRM email automation

Status: development required. This is separate from section 9's Supabase SMTP configuration.

14.1 Prepare the sending identity

1. Verify the intended sender/domain in Brevo.
2. Complete the provider's domain authentication and review SPF/DKIM/DMARC alignment as appropriate to the domain's existing mail setup.
3. Obtain a Brevo **API key** for API access and store it in the backend's secret storage. Do not use the SMTP key as an API credential.
4. Decide which workspace may use which sender, domain, templates, and provider connection.
5. Separate transactional messages from marketing campaigns, including their consent and unsubscribe requirements.

Using a transactional endpoint does not make promotional content exempt from marketing rules. An imported email address is not opt-in evidence.

14.2 Implement a Brevo adapter

The existing CRM's email /send contract needs an adapter that validates the user/workspace/recipient, chooses the authorized sender, creates a durable job, and calls Brevo. As documented by Brevo, transactional email uses:

TEXT

```
POST https://api.brevo.com/v3/smtplib/email
api-key: <BACKEND-ONLY-BREVO-API-KEY>
Content-Type: application/json
```

Illustrative template payload, **not a request executed by this guide**:

JSON

```
{
  "sender": { "name": "Example Business", "email": "team@example.com" },
  "to": [{ "email": "recipient@example.com", "name": "Example Contact" }],
  "templateId": 42,
  "params": {
    "FIRSTNAME": "Example",
    "BUSINESS_NAME": "Example Business",
    "APPOINTMENT_TIME": "A confirmed date, time, and timezone"
  }
}
```

All addresses are reserved examples and 42 is an illustrative template ID. Replace them only in a controlled, authorized test with your own verified sender/template and a permitted recipient. Parameter names must match the chosen Brevo template.

CRM template UUIDs and Brevo numeric template IDs are not interchangeable. Add an explicit workspace-scoped mapping. Alternatively render validated CRM content server-side into the documented HTML/text request format. Do not assume saving a CRM template synchronizes it with Brevo.

The current manual email composer sends only `lead_id`, `channel`, and `body`; it does not pass a subject or a selected provider template ID. Either define an appropriate backend subject/content policy for that existing

contract, or deliberately extend and test the client/server contract for template-based email. Existing template subjects are not evidence that a sender already delivers them.

Persist the returned provider message identifier and the corresponding CRM message. The current frontend expects a saved message object from `/send`, not just an HTTP success flag. Add server-generated idempotency protection for email; the current email client payload does not automatically include the WhatsApp request-ID behavior.

14.3 Track outcomes and suppression

Configure Brevo transactional/marketing webhooks for the event types your implementation supports. Authenticate callbacks with a supported mechanism such as configured bearer authorization; do not assume Meta's webhook signature scheme applies to Brevo.

Associate each event with the persisted provider message and workspace. Deduplicate callbacks and handle delivery, bounce, blocked, complaint/spam, and unsubscribe events. Stop future prohibited sends promptly and reconcile state with the provider where needed.

Email API acceptance means the provider accepted a request, not that the inbox received it. Opens/clicks are imperfect signals because mail clients and privacy tools may preload or suppress them; do not use an open event as proof of consent or a completed sale.

14.4 Test before enabling a campaign

- Verify actual receipt at controlled addresses and inspect sender identity and links.
- Test invalid addresses, expired credentials, missing template parameters, and provider throttling.
- Verify suppression after unsubscribe, complaint, and relevant bounce events.
- Confirm that retries and duplicate callbacks do not generate duplicate messages.
- Test wrong-workspace sender/template access and absence of secrets in responses/logs.
- Test pause/resume and an uncertain send without blind replay.

Provider references: [transactional email](#), [webhook events](#), and [secured webhooks](#).

CHAPTER 15

Create Make automations

This chapter contains the complete Make.com creation walkthrough inside the handbook. A standalone copy is retained for focused use.

This walkthrough explains how to create and manage Make scenarios for lead intake, internal alerts, WhatsApp Business API messages, and Brevo email workflows.

Status boundary: Make orchestration is separate from the CRM. The shipped Cloudflare gateway only checks health and access. It does not receive leads, send messages, or run jobs. The `/integrations/make/...` endpoints used below are **proposed integration contracts that a developer must implement and test**, not URLs already available in SaSSy CRM. Do not activate a scenario against an unimplemented endpoint.

The repository separately documents a property-specific Make -> Supabase -> Gmail integration. That is not automatically installed for new workspaces and is not a Brevo/WhatsApp sender. This guide does not change that existing scenario or copy its customer IDs and credentials.

15.1 Choose the workflow and integration boundary

Start with one business, one trigger, and one narrowly defined outcome. Examples:

- Capture a Facebook Lead Ads submission as a CRM lead.
- Notify the responsible staff member about a new enquiry.
- Send an appointment reminder to a consenting customer.
- Send an approved follow-up sequence, stopping when the person replies or opts out.

For a shared SaaS product, the recommended division of responsibility is:

TEXT

Make	SaSSy integration backend	Provider
Acquire event ----->	Validate source and workspace	
Transform fields ----->	Save lead once	
Request approved action --->	Check consent and create job	
Run scheduled trigger ----->	Claim due jobs and send ----->	WhatsApp / Brevo
	Store delivery outcome <-----	Authenticated webhook

Make handles orchestration. The backend owns tenant isolation, provider secrets, consent, durable jobs, and delivery records. An editable Make workflow should not be able to choose an arbitrary workspace or bypass these checks.

If Make instead calls a provider directly, it becomes part of the trusted delivery system. You still need server-side eligibility checks, durable attempt tracking, and reconciliation. A simple filter plus Data Store is not enough for a reliable multi-tenant campaign engine.

15.2 Prepare accounts and backend prerequisites

Accounts and access

- Make account/team with an appropriate plan and access controls.
- Intended SaSSy workspace and permission to operate its integrations.
- Meta account with access to the relevant Page, lead form, and leads when using Facebook Lead Ads.
- WhatsApp Business Platform sender/account and approved templates when sending WhatsApp messages.

- Brevo account with a verified sender/domain for email.
- A staging environment and destinations you control for tests.

An ordinary CRM team member should not need Cloudflare tokens or the Supabase service-role key. Only authorized operators should maintain integration credentials and scenario settings.

Developer prerequisites for the recommended scenarios

Before connecting Make, implement and document a machine-to-machine interface. Suggested contracts:

Proposed endpoint	Purpose	Required behavior
POST /integrations/make/intake	Validate a source event and persist the lead	Resolve workspace from a trusted route and scoped credential; deduplicate source IDs; return a persisted lead ID
POST /integrations/make/enqueue	Request an approved action for a lead	Validate lead/workspace, policy, template, consent, recipient, and idempotency key; create at most one logical job
POST /integrations/make/dispatch-due	Ask the backend to process a bounded batch	Server-side schedule checks, job claims/leases, dispatch validation, provider call, and durable results
GET /integrations/make/runs/:id	Inspect a run, if asynchronous execution is used	Authenticate and scope the run; distinguish queued, accepted, failed, and unknown outcomes

These routes and their machine credentials do not exist in the included gateway. The existing browser /send and /automation/run routes also require implementation; a Make API key cannot be substituted for a user's Supabase token without designing that authorization path.

Use a distinct, revocable integration credential bound server-side to the permitted business and actions. Store it through Make's dedicated connection/credential facilities, not inline in JSON bodies, scenario names, logs, or shared blueprints. Reject an event whose Page/form does not belong to an approved route.

The backend should provide stable responses such as the following. This is an **example contract**, not the response of an existing generic route:

JSON

```
{
  "ok": true,
  "created": true,
  "lead_id": "EXAMPLE-CRM-LEAD-UUID",
  "event_key": "meta:PAGE:FORM:LEAD"
}
```

On a replay, return the same lead ID with `created: false`, without overwriting later sales edits. A separate enqueue request must be independently idempotent, so recovery still works if the first intake response was lost.

15.3 Create your first Make scenario

1. Sign in to the intended Make organization/team.
2. Open **Scenarios** and choose the create-new-scenario action.
3. Name it clearly, for example **SaSSy | Example Business | Meta intake | STAGING**.
4. Leave automatic scheduling off while building.
5. Add the first trigger module using the plus button and search for the required application.
6. Create/select the intended connection. Confirm the account, business assets, and requested permissions.
7. Add later modules one at a time and map from real test output rather than guessing field paths.

8. Save the scenario after each meaningful change.
9. Use **Run once** only with controlled data and with downstream sending disabled or pointed at controlled recipients.
10. Inspect module input/output, filters, and execution history before adding automatic scheduling.

Make module labels and fields can change by app/version. The module names below are functional guides; use the equivalent currently shown in your account and verify its input/output.

Important: Run once can execute writes and send real messages. It is not automatically a dry run. Disable sending routes or use a backend's genuine validation-only mode first.

15.4 Scenario A: Facebook leads into SaSSy CRM

Module layout

TEXT

```
Facebook Lead Ads: Watch Leads
-> Filter: expected form and minimum fields
-> JSON: Create JSON
-> HTTP: POST to trusted intake endpoint
-> Filter: successful persisted result
-> Optional HTTP: enqueue approved staff alert
```

Module 1 - Watch Leads

1. Add **Facebook Lead Ads -> Watch Leads**, or the current supported polling equivalent.
2. Connect the authorized Meta account.
3. Select the exact Page and form. Confirm the account has leads access, not merely Page visibility.
4. Choose a conservative test limit and an appropriate starting point. Do not select all historical leads unintentionally.
5. Create a permitted synthetic form submission or use a retained test that you are authorized to replay.
6. Run the trigger in isolation and inspect the returned bundle.

Make also lists a **New Lead** trigger for its Facebook integration. If choosing an instant/webhook mode, follow its current setup requirements and verify acquisition. Polling and instant triggers have different behavior; do not enable both for the same source without stable deduplication.

A form may need a prior submission before it is available for retrieval. Do not delete real leads or repeatedly replace retained test submissions merely to make a picker refresh.

Filter - accept only the intended source

Add a filter on the connection after the trigger:

- Page/form matches the approved source for this scenario.
- External lead ID exists.
- Submission timestamp and required contact fields exist.
- Test data is clearly identified during staging.

Send invalid records to a restricted diagnostic path or stop with a clear error. Do not invent missing names, dates, or consent to pass the filter. Backend validation remains mandatory even when the Make filter is correct.

Module 2 - Create JSON

Add **JSON -> Create JSON** and define a data structure agreed with the backend developer. Example normalized fields:

Logical field	Source/mapping	Validation
source	Fixed meta_lead_ads	Do not accept an arbitrary source type from submitted answers
external_id	Actual Meta lead identifier	Preserve as text; never convert long IDs to floating-point numbers
page_id, form_id	Verified Meta output/configuration	Must match a server-approved route
submitted_at	Original provider creation time	Include timezone; do not replace it with each retry time
name	The form's full-name answer	Required if the business/backend requires it
phone	Form phone answer	Preserve input; normalize/validate with known country context server-side
email	Form email answer, if present	Do not fabricate it for a phone-only form
answers	Approved custom answers	Map field by field; match CRM custom field keys

Inspect whether your trigger returns a field directly or inside an answer array. Use the relevant answer value, not the whole bundle. Do not copy module numbers from screenshots: Make assigns different IDs in different scenarios.

Use the JSON module so quotation marks, newlines, and backslashes are escaped properly. Hand-building JSON with pasted customer text can break requests or alter their structure. Do not include credentials, workspace IDs chosen by respondents, roles, ownership overrides, or unverified consent flags.

Module 3 - Send to the trusted backend

Add **HTTP -> Make a request** or the current equivalent:

Setting	Value
Method	POST
URL	Your implemented HTTPS intake endpoint, for example /integrations/make/intake on your backend origin
Authentication	The scoped machine credential agreed with the backend developer
Content type	application/json
Body	The output string/object from Create JSON, according to the HTTP module's body mode
Parse response	Enabled when available, so response fields can be mapped

Use HTTP's dedicated **Credentials** configuration for secrets. Configure the required authentication header through that facility. Do not place a secret in the URL query string or ordinary body fields.

Run with one synthetic event. Confirm the returned lead ID corresponds to exactly one record in the intended workspace. Replay the same event and confirm the same ID is returned without duplicate records or lost staff edits.

Module 4 - Optional internal notification

After successful persistence, call the proposed enqueue endpoint to request a named, pre-approved staff notification. The backend resolves its permitted recipient and template; the form submitter must not be able to select an arbitrary notification address.

Use a stable logical key such as:

TEXT

```
business-route + source + page + form + external-lead-id + internal-alert-v1
```

Do not filter only on `created = true`. If Supabase saved the lead but its first response was lost, intake retry may return `created = false` even though the first alert still needs to be queued. Intake deduplication and notification idempotency must be separate.

Verify actual receipt of the controlled internal alert, not just a green HTTP module. An internal staff alert does not opt the lead into marketing.

15.5 Scenario B: WhatsApp and Brevo follow-ups

Prerequisite: the backend implements provider adapters, scheduling/eligibility, consent, outbox processing, and the proposed Make authorization endpoints. This is not enabled by connecting the health-only Worker.

Recommended scheduled scenario

TEXT

```
Make schedule
-> HTTP: POST /integrations/make/dispatch-due
-> Router: completed / accepted for processing / needs review
+> Restricted operations alert for actionable failures
```

The backend should select and claim due jobs atomically. Do not have several Make runs independently query “unsent” rows and send them without claims; both may dispatch the same message.

Configure the request

1. Create **SaSSy | Example Business | Due follow-ups | STAGING**.
2. Add an HTTP request authenticated with that business's scoped integration credential.
3. Use the backend's documented body, such as a bounded batch limit and a real `dry_run` option **only if implemented**.
4. The backend chooses eligible jobs; Make does not submit an arbitrary workspace or provider token.
5. Parse the response and route based on its actual status. If it only accepts a job, use a supported run-status endpoint or backend monitoring; do not label acceptance as delivery.
6. Test with one due appointment, one cancelled appointment, one opted-out contact, and a duplicate trigger.
7. Confirm only the eligible controlled destination receives a message.

Choosing WhatsApp or Brevo

Channel selection should be an approved policy, not an uncontrolled fallback:

- **WhatsApp:** use that business's authorized sender and permitted approved template or valid service-window message.
- **Brevo email:** use the mapped verified sender and appropriate transactional/marketing template.
- **No permitted channel:** skip safely or create a staff action; do not message without consent to keep the scenario green.

- **WhatsApp failed:** do not automatically send email unless the customer is separately eligible for that email and the failure policy permits it.

For a visible Make Router, branch on the backend's validated channel/eligibility response. Both routes should call approved backend actions. Never rely on a user-editable field such as `can_send=true` without revalidation on the server.

If the provider call must run inside Make

This is an alternative delivery design, not a shortcut around backend requirements:

1. Claim a job through a trusted backend, obtaining a bounded, validated delivery specification.
2. Use a connection restricted to the correct business's provider account.
3. For WhatsApp, call Meta's supported versioned message API with the correct phone/account context, approved content, and recipient.
4. For Brevo, use its current native module if it supports the required fields, or HTTP POST `https://api.brevo.com/v3/smtp/email` with an API-key credential.
5. Immediately persist provider acceptance/message ID through an authenticated job-result endpoint.
6. Handle actual delivery through authenticated provider webhooks.
7. Mark a lost/uncertain send response for reconciliation. A later trigger must not blindly repeat it.

Job-claim and job-result endpoints, leases, provider credential isolation, and ambiguity handling must be designed and implemented. They are not part of the base CRM. For the current project, keeping provider dispatch inside the backend is the simpler security boundary.

15.6 Scenario C: a controlled Brevo internal-alert pilot

This explains a small Make-native email test after **verified CRM ingestion**, using one approved internal recipient. It is not the production architecture for customer marketing or a guarantee of exactly-once delivery.

Modules

TEXT

```
Successful CRM intake result
-> Data Store: check for alert marker
-> Filter: marker does not exist
-> JSON: create Brevo email body
-> HTTP: Brevo transactional email API
-> Data Store: record accepted-message marker
```

Build the email request

1. Verify your business's sender/domain in Brevo and obtain its API key.
2. Store that key in Make HTTP's dedicated credential facility, configured for the `api-key` header.
3. Create a JSON structure containing a fixed authorized sender, a fixed internal recipient, subject, and text or HTML content.
4. Prefer plain text initially. If HTML includes lead data, escape that data; JSON escaping is not HTML escaping.
5. Set HTTP method POST, URL `https://api.brevo.com/v3/smtp/email`, content type `application/json`, and the JSON module's output as the body.
6. Parse the provider response and save the returned message identifier with the event marker.
7. Verify actual receipt and inspect Brevo delivery logs. The accepted marker must not be described as "delivered."

Suggested internal subject: **SaSSy CRM | New enquiry for Example Business.**

Suggested plain-text content:

TEXT

A new enquiry has been saved in the CRM.

Name: [mapped name]

Contact: [mapped phone or email, if supplied]

Requirement: [approved mapped answer]

Submitted: [original date/time with timezone]

Open your CRM workspace to follow up.

Map these bracketed placeholders in Make. Do not leave them literal, paste real customer data into a reusable blueprint, or invent a direct lead URL; the current app does not provide a supported public deep link to an individual lead.

Data Store setup

Create a Data Store dedicated to this pilot with a stable event/action key. Useful fields include `provider_message_id`, `accepted_at`, and an explicit status such as `accepted` or `review_required`.

Use **Check the existence of a record** or the current equivalent, then filter on a missing key. After confirmed provider acceptance, write/update the marker. Keep sequential processing enabled for this pilot to reduce overlap, but do not mistake it for a transaction or cross-scenario lock.

Critical limitation: the email can be accepted before its marker is saved. If the marker write fails, another run may send a duplicate. If the email response itself is uncertain, stop and reconcile it rather than letting an automatic retry send again. Inspect provider logs before replay; resume only the failed marker write when acceptance is known. A proper outbox/lease/reconciliation system is required for broader use.

Never use a pre-send “sent” marker to hide the issue; that can lose alerts when the send fails. Do not run this pilot's direct-email route and the backend's notification route for the same logical event.

15.7 Schedule and activate safely

1. Keep the scenario off while editing and during initial Run once tests.
2. In scenario settings, enable **Store incomplete executions** if available and appropriate. Make documents this as disabled by default.
3. Choose sequential/ordered processing when event order matters, especially for a marker-based pilot. Understand how unresolved failures may affect later processing.
4. Do not silently discard unfinished work when storage is full. Monitor capacity and plan recovery.
5. Open the scheduling control and choose an interval supported by your Make plan and provider quotas. For lead polling, 15 minutes can be an example starting point, not a guaranteed available or required setting.
6. Check the timezone and any active time window. Authentication, cron timing, and customer quiet hours are separate concerns.
7. Save and enable scheduling only after the correct connections and test destinations have been reviewed.
8. Observe a genuinely scheduled run using a controlled new event. A manual replay alone does not verify acquisition or scheduling.
9. Confirm the CRM record, message identity, actual receipt where applicable, and duplicate behavior.
10. Document the owner, failure alert destination, provider costs, and how to stop dispatch.

Stopping Make scheduling prevents future scheduled invocations. It may not cancel an already-running scenario or jobs already queued in the backend. Pause the backend delivery policy and cancel/review pending jobs separately when required.

15.8 Duplicate protection and error recovery

Keep three identities separate

Identity	Purpose	Example shape
Source event ID	Save a provider submission once	Meta Page + form + lead ID
Logical action key	Queue one intended message/action	Source event + action/template version + channel
Provider message ID	Track the actual send and callbacks	ID returned by Meta/Brevo

The same phone number can make two genuine enquiries. Deduplicating by phone alone can lose one. Conversely, giving a retry a new event ID can create duplicates.

Configure error handling deliberately

Failure	Response
Invalid/missing field or unapproved source	Stop that item, retain a safe diagnostic, correct the mapping/route; do not keep resending unchanged bad input
Authentication or forbidden access	Pause affected processing and fix the authorized connection; do not switch to a broad shared key
Idempotent intake request times out	Retry with the same external event ID; inspect the existing CRM record
Provider rate limit	Respect provider retry guidance with bounded backoff and preserved job identity
Provider send may have been accepted	Mark unknown/review required and reconcile before another send
Provider acceptance confirmed, marker/result save failed	Retry only the result/marker persistence, not the send module
Permanent recipient/template failure	Record a terminal safe failure and notify the responsible operator; do not retry forever
Duplicate delivery webhook	Deduplicate and apply state safely; do not create another outbound message

Make supports incomplete executions and retries, but retry behavior must be evaluated per module. Automatic replay is appropriate only when the operation is safe to repeat. Do not attach a blanket retry handler to a non-idempotent email/WhatsApp send. Do not use Ignore to turn a failed CRM save or send into apparent success.

When reviewing an execution, record the time/timezone, module, source event ID, response status, and provider message ID if available. Keep secrets and unnecessary contact content out of incident messages and shared screenshots.

15.9 Maintain the existing property-specific workflow

The separate Meta intake runbook records a historical chain of Facebook polling, JSON transformation, a trusted Supabase RPC, a Data Store check, Gmail, and a marker write. It is scoped to one business and form configuration. Its historical “Active” state is not live-verified by this guide.

If authorized to maintain that installation:

1. Read its runbook and inspect the current saved scenario before changing it.

2. Confirm the exact connected Page/form and approved destination. Do not copy a customer's IDs from another scenario.
3. Confirm the required additive migrations already exist. Do not rerun `saas.sql` or the full migration files to repair a Make mapping.
4. The existing trusted function expects a JSON object named `p_payload` when called through Supabase's RPC HTTP interface:

TEXT

```
POST https://YOUR-PROJECT.supabase.co/rest/v1/rpc/ingest_meta_lead
```

5. Its allowed payload fields include `meta_lead_id`, `page_id`, `form_id`, `submitted_at`, `full_name`, `phone_number`, and `enquiry_intent`. Full-form routes also require the property-specific answers defined in the migration. IDs and answer values are text; timestamps retain timezone information.
6. This RPC is restricted to the trusted server role. A public publishable key or ordinary CRM user's bearer token cannot call it. Preserve the operator-owned secure connection; never expose its elevated credential to customers or paste it into public instructions.
7. The route determines the workspace and validation variant server-side. Do not add caller-selected workspace, role, owner, or consent fields to bypass it.
8. The transformer contains property-specific custom fields and Delhi/INR defaults. Do not reuse it unchanged for a general SaSSy customer.
9. Preserve source IDs during retries. The CRM record and internal email marker have independent duplicate logic.
10. Verify trigger coverage and a real scheduled acquisition. An enabled database route is not proof that an old or new form is being polled.

Changing the internal alert from Gmail to Brevo is a separate provider migration: verify sender/recipient, test with controlled data, preserve marker identity, and disable the old sending path before enabling the new one. Do not run both and accidentally send two alerts. Review uncertain in-flight sends before cutover.

15.10 Handover checklist and references

Before turning on a scenario

- [] Named owner, business, purpose, and environment.
- [] Correct Make organization/team and minimal connection permissions.
- [] Correct Page/form/trigger and deliberate starting point.
- [] Backend endpoints actually implemented; no placeholder URL or health-only gateway used as a sender.
- [] JSON mappings checked with real-shaped synthetic test bundles, including quotes and optional fields.
- [] Scoped credential configured in dedicated secret/connection fields.
- [] Server-side workspace routing, consent, suppression, and action authorization verified.
- [] Same-event replay tested; one lead and one logical action remain.
- [] Actual controlled-recipient delivery verified, separately from provider acceptance.
- [] Error routes, uncertain-send recovery, capacity limits, and alerts reviewed.
- [] Schedule/timezone/costs approved; scheduled acquisition observed.
- [] Pause procedure covers Make, already-running executions, and queued backend jobs.
- [] Shared blueprint/screenshots contain no credentials, personal data, or private invitation links.

Keep a small operations record

Record the scenario name/link, business, purpose, connection owners, intended source, destination backend, action/template version, schedule/timezone, last controlled test, alert recipient, and rollback/pause instructions. Do not include secret values.

Official documentation

These pages were retrieved while preparing this walkthrough. UI labels, plan limits, and module versions can change; check the current documentation and your own account before activation.

- [Make: create your first scenario.](#)
- [Make: Facebook Lead Ads modules.](#)
- [Make: HTTP and credential configuration.](#)
- [Make: scenario scheduling.](#)
- [Make: Data Stores.](#)
- [Make: incomplete executions.](#)
- [Make: automatic retry of incomplete executions.](#)
- [Brevo: transactional email API.](#)
- [Meta: WhatsApp setup.](#)
- [SaSSy CRM integration contracts.](#)

15.11 Adapting the pattern to n8n

The same security and reliability boundaries apply to an n8n implementation: a trusted trigger, validated fields, a scoped integration connection, server-side workspace routing, idempotent jobs, and authenticated delivery callbacks. n8n node names, credential setup, retries, and scheduling must be configured and tested separately; a Make scenario cannot simply be pasted into n8n. Neither tool is provisioned by the CRM's Connections screen.

CHAPTER 16

Example automation recipes

These are recipes to implement after the integration prerequisites, not preinstalled rules. Timings below are example business choices, not existing schedules or guarantees.

Recipe	Trigger and conditions	Actions	Stop/safety conditions
New-enquiry acknowledgement	Valid new lead, approved source, appropriate channel consent	Create lead, assign owner, queue approved WhatsApp template or Brevo email	Duplicate event, missing consent, suppression, invalid address, or paused workflow
Appointment reminder	Confirmed appointment approaching, valid timezone and permitted communication	Queue a reminder using the correct date/time and template	Cancellation, reschedule, prior reminder, opt-out, or changed recipient
Overdue follow-up alert	Lead's follow-up is overdue and still open	Notify its responsible staff member through a configured internal channel	Closed/deleted lead, inactive assignee, resolved follow-up, or already-alerted occurrence
Lead nurture sequence	Explicit enrollment, documented marketing consent, approved content	Example: introduction, helpful resource, then optional follow-up on selected days	Reply requiring human attention, opt-out, closure, frequency cap, consent withdrawal, or pause
Stage-change handoff	A record moves to an agreed stage	Create a staff action or queue a suitable transactional message	Duplicate transition, unauthorized stage change, missing recipient, or stale job

Worked example: appointment reminder

1. A staff member records a confirmed appointment and its timezone.
2. The backend derives a reminder job with a unique key such as workspace + appointment + reminder type + appointment version.
3. The job becomes due at the chosen lead time, for example one day before the appointment.
4. The delivery worker rechecks that the appointment is still confirmed, has not moved, the lead remains active, consent is valid, and the workflow is not paused.
5. It sends an approved WhatsApp template or an appropriate Brevo email through the mapped provider connection.
6. It stores the provider ID and later updates the outcome from authenticated callbacks.
7. Cancellation or rescheduling invalidates the old pending job. A timeout with uncertain acceptance is reconciled before any resend.

Business approval checklist for every recipe

- Who owns the workflow, and who may enable or pause it?
- What exact event enrolls a person, and what excludes them?
- Which workspace, sender, template, channel, and timezone apply?
- What permission/consent evidence supports the message?
- What is the frequency cap and quiet-hours policy?
- What ends the sequence: response, conversion, deletion, or opt-out?
- How will duplicate events and uncertain delivery be handled?
- Who receives an error alert, and how can pending dispatch be stopped?

CHAPTER 17

Operate and maintain the platform

Business Owner routine

Daily: review unassigned/overdue leads, upcoming meetings, incomplete notes, and actual delivery errors if a sender is implemented.

Weekly: review pipeline progress, inactive opportunities, team workload, source quality, duplicate imports, and pending invitations. Improve data quality before adding more automation.

When staff change: update workspace access and separately remove access to external provider and automation tools. Reassign open work deliberately.

Platform operator routine

Frequency/event	Checks
Regularly	Sign-in availability, authentication email delivery, database health, error logs, storage/usage, and service incidents
After provider integration	Queue age, failed/unknown sends, webhook processing, suppression updates, token expiry, quota and spend
On access changes	Operator roles, secrets access, shared automation connections, and tenant-isolation implications
Before a release	Staging verification, migrations, dependencies, correct public configuration, privacy content, and rollback plan
At a planned interval	Backup/restore drills, access review, retention cleanup, secret rotation, and incident-response rehearsal

Choose backup and recovery facilities suitable for the actual Supabase plan and business requirements. Document recovery objectives and test restoration in an isolated environment. Lead exports are not a replacement for database backups.

Secrets and logs

- Use server-side secret facilities for provider tokens, SMTP credentials, app secrets, and elevated database keys.
- Do not put secrets in VITE_ variables, browser storage, health responses, screenshots, or customer-accessible settings tables.
- Separate staging and production credentials and destinations.
- Limit execution-log access; logs can expose contact details, message contents, and invitation links.
- Rotate compromised credentials, revoke old access, and review affected actions. Do not merely remove a leaked key from the latest source file.
- Cloudflare secret operations can publish a new Worker version; follow the current deployment process rather than assuming secret updates are inert.

Data protection and customer trust

Publish product-appropriate privacy/terms documents, document processors and retention, handle deletion/export requests, and restrict what personal data is collected. Keep customer data out of public demos and screenshots. Neither the presence of RLS nor passing application tests is a compliance certification or an independent security audit.

Commercial launch work still required

Before charging customers, design and implement any required subscriptions/payments, usage quotas, platform administration, provider onboarding and secret management, abuse prevention, customer support, and operational monitoring. Review legal, tax, billing, and data obligations appropriate to where you operate.

CHAPTER 18

Troubleshooting

Symptom	Likely explanation	Safe next step
Setup asks for credentials you never created	Database configuration and account creation are separate; possibly standalone mode	Confirm intended SaaS mode/schema, then use Create an account; there is no default password
Verification email never arrives	SMTP/sender restrictions, filtering, rate limiting, or incorrect address	Check spam and the address; operator inspects Supabase/Brevo logs and SMTP configuration
Verification opens localhost or the wrong site	Incorrect Site URL/redirect configuration	Operator corrects exact production URLs; request a fresh link and do not disable verification
Connection test fails on an empty database	CRM schema/marker is absent	Inspect the target; install the correct schema only if it is genuinely new and empty
Connection test says wrong schema	SaaS/standalone mismatch	Confirm intended architecture; do not run the other full schema over the existing database
Connection fields are locked	Hosted infrastructure is deliberately operator-managed	Workspace users should continue with accounts/workspaces; operator changes the build configuration if needed
A local connection works only in one browser	Browser-local setup is not deployment configuration	Configure public production build variables and rebuild for all visitors
No business records are visible	Wrong workspace, filters, missing membership, or a failed request	Confirm workspace, clear relevant filters, and check active access before changing data
Invitation cannot be accepted	Wrong/unverified email, expiry, revocation, or already used link	Sign in with the invited verified email; ask the Owner for a new link if necessary
Invitation link was lost after reload	Raw invitation token is not retained for redisplay	Owner revokes/replaces the invitation and securely copies the new link
Team update conflicts	Another administrator changed the member	Refresh and review the current access before retrying
WhatsApp/Brevo appears disconnected	Sender backend not implemented/configured, or provider health failed	Check the actual backend and provider; a saved URL is not sufficient
Gateway reports connected but sending fails	Health-only gateway has no sending routes	Implement/test the required integration endpoints; do not falsify health flags
An automation switch fails or no rules appear	Missing backend routes, no provisioned rules, or unavailable provider	Review rule provisioning and execution service; demo rules do not automatically exist in live mode
A Meta form is mapped but no leads arrive	No trigger coverage, expired provider connection, failed/pending Make runs	Check the exact active form and scenario history; a route alone is not a listener
Brevo login emails work but CRM campaigns do not	Supabase SMTP and CRM API delivery are separate	Implement the CRM sender; do not reuse SMTP credentials as an API key

Symptom	Likely explanation	Safe next step
Message status is uncertain	Provider response was lost or callback is pending	Reconcile provider logs/IDs before retrying; avoid duplicate sends
Imports create duplicates	Import is create-only; previous rows may already have saved	Inspect saved records; use same-dialog continuation for remaining rows rather than restarting the full file
Call button is absent	Desktop layout, missing/invalid saved number, or stale assets	Check the saved number and phone layout; refresh after a release
Theme differs on another device	Appearance is browser-local	Set Appearance separately on that device
Export contains fewer records than expected	Export respects current filters	Review filter scope; do not assume export is a complete backup
New deployment still shows old behavior	Wrong project, wrong build configuration, or stale app assets	Check the deployed release and fresh browser context; do not clear all storage before saving work

When reporting a problem, include the relevant workspace, time/timezone, action, safe error message, and whether it occurs in another browser. Redact passwords, tokens, private invitation links, and unrelated customer information.

CHAPTER 19

Verification and launch checklist

Existing local verification commands

From the CRM root, these commands exercise the existing test/build scripts. Browser tests use isolated fixtures/mocks and do not prove real provider delivery.

POWERSHELL

```
npm.cmd test
npm.cmd run build
```

If browser binaries are not installed, install the project's Playwright Chromium dependency, then run the relevant suites:

POWERSHELL

```
npm.cmd playwright install chromium
npm.cmd run test:browser
npm.cmd run test:saas
npm.cmd run test:settings
npm.cmd run test:theme
npm.cmd run test:accent
npm.cmd run test:call
```

The Meta ingestion extension has an isolated test suite:

POWERSHELL

```
node --test tests/meta-lead-ingestion.test.js
```

Run only reviewed migration/test SQL in an appropriate disposable environment. Do not point destructive test fixtures at customer data. The commands are supplied as a runbook; this documentation task did not execute the application suite or redeploy the product.

Before accepting customer data

- [] Correct operating mode and schema, with existing data preserved.
- [] Public build configuration points to the intended environment.
- [] Exact authentication URLs and production email delivery tested.
- [] Two-workspace isolation, wrong-workspace writes, and revoked access tested.
- [] Owner/invitation/member workflows tested with controlled accounts.
- [] Dependencies/security reviewed, including the existing spreadsheet parser warning.
- [] Backup and restore process tested; recovery ownership is documented.
- [] Privacy/terms/retention and support processes appropriate to the product.
- [] Hosting and database monitoring, rate limits, and an incident contact are in place.
- [] Public frontend assets contain no private credentials or customer exports.

Before enabling an automation

- ☐ Required backend routes and storage are implemented, not only UI controls.
- ☐ Provider ownership and workspace-scoped credentials are explicit.
- ☐ Consent, suppression, template status, timing, and recipient validation enforced at dispatch.
- ☐ Stable event IDs and duplicate protection tested.
- ☐ Callback authentication, replay handling, and truthful delivery states tested.
- ☐ Permanent failures and ambiguous sends have separate recovery paths.
- ☐ Provider rate limits, usage costs, batch sizes, and pause controls reviewed.
- ☐ Controlled-recipient delivery verified; no live customer blast used as a smoke test.
- ☐ Run history and alerts identify who will resolve failures.
- ☐ Workflow starts with a small approved pilot and has a clear stop condition.

CHAPTER 20

Developer map and references

Source map

Area	Source
Main navigation and workspace application	src/App.jsx
Account and workspace entry	SaaSRoot.jsx, Login.jsx
Supabase client and selected workspace	supabase.js, connections.js, tenant.js
Workspace membership operations	saas.js, SaaSTeam.jsx
UI roles and permissions	auth.js
CRM data and integration client contracts	db.js
Business presets/customization	workspace.js, WorkspaceSetup.jsx
Settings and connections	Settings.jsx, ConnectionSettings.jsx
Lead management and pipeline	Dashboard.jsx, LeadProfile.jsx, Funnel.jsx
Import/export	ImportModal.jsx, xlsx.js
Mobile calling	CallLeadButton.jsx, phone.js
Appearance	AppearanceSettings.jsx, AccentSettings.jsx, palette.js
Fresh SaaS database	supabase/saas.sql
Health-only Worker	cloudflare/src/index.js, wrangler.jsonc

Project documentation

- README and installation overview.
- Integration route contracts and security boundaries.
- Make.com automation creation walkthrough.
- Cloudflare gateway deployment.
- Historical hosted deployment notes; verify current cloud state before relying on dated status.
- Data migration guidance.
- Business-specific Meta ingestion history; not a generic customer onboarding procedure.

Official provider references

The relevant pages were retrieved while preparing this guide on 2 October 2026. Provider dashboards, policies, API versions, and availability can change; check the current page before implementation.

- [Cloudflare Pages Direct Upload](#).
- [Cloudflare Worker secrets](#).
- [Supabase custom SMTP](#).
- [Meta WhatsApp getting started](#).
- [Meta messaging and service windows](#).

- Brevo SMTP integration.
- Brevo transactional email API.
- Brevo transactional webhooks.
- Brevo webhook authentication.

Recommended rollout order

1. Stabilize core CRM setup, verified signup, workspace isolation, and everyday use.
2. Configure production authentication email and operational safeguards.
3. Implement and test one provider connector for one controlled workspace.
4. Add durable scheduling, delivery reconciliation, and a small approved automation pilot.
5. Build self-service provider onboarding and commercial SaaS capabilities only after these foundations are verified.